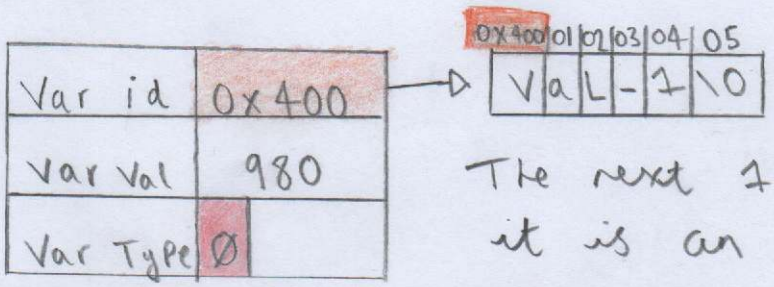
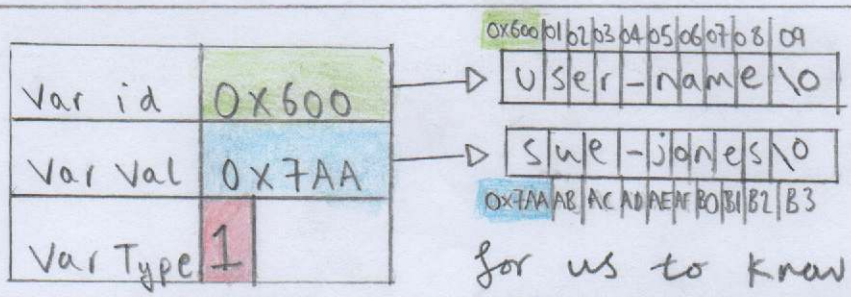
VARIABLES AND DATA TYPES

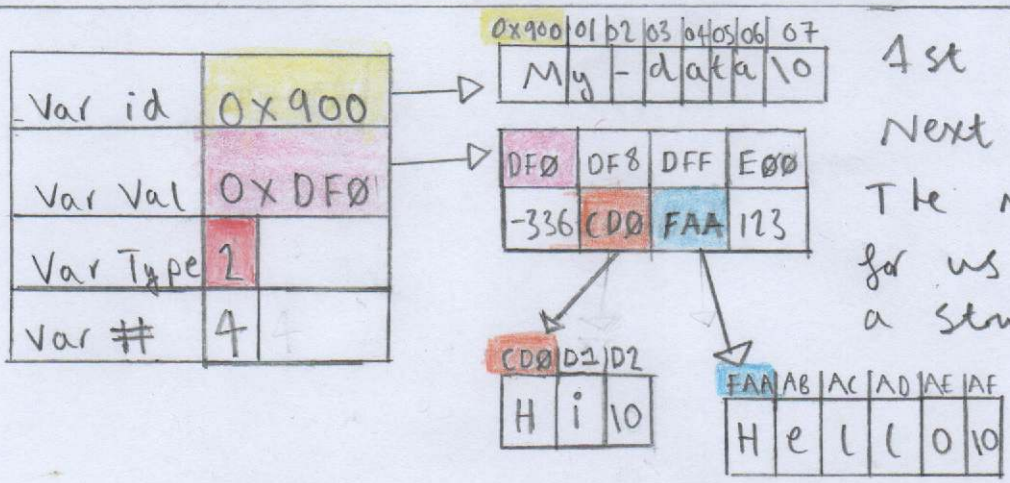
I made 3 data types (signed int, strings and struct). Strings are always terminated, even for a single char. Structs are user defined variables, like a struct in 'C', that means they can store strings, ints and pointers to other structs. All of these data types are stored in the heap segment and I put 1 byte value that will tell me if it's a string (#1), int (#0) and struct (#2), here is some examples:



1st 8 bytes for the Id.
Next 8 bytes for the value.
The next 1 byte is for us to know it is an integer.



1st 8 bytes for the Id.
Next 8 bytes for value.
The next 1 byte is for us to know it is a string.

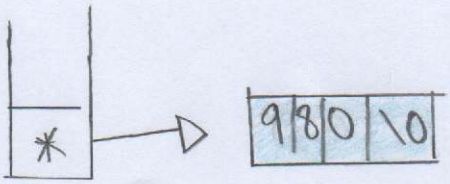


1st 8 bytes for the Id.
Next 8 bytes for the value.
The next 1 byte is for us to know it is a struct. Last byte is for us to know the number of vars.

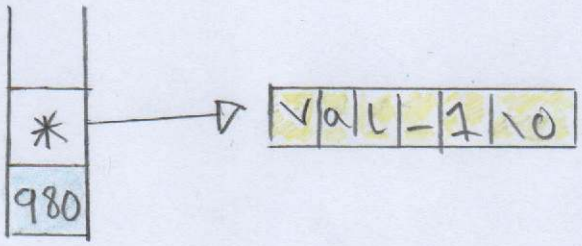
The 1st Var is Val-1, the byte is set to 0 = integer.
The 2nd Var is user-name, the byte is set to 1 = String.
The 3rd Var is my-data, the byte is set to 2 = Struct.

Every time a variable is created, I place it on a variable stack and I called this dict with a dict-tos pointer to the top.

In my input file: 980 int val-1 def-i

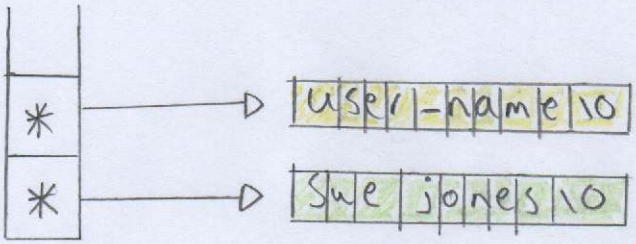


Operand int will cast the string at the top of the run time stack to an int.



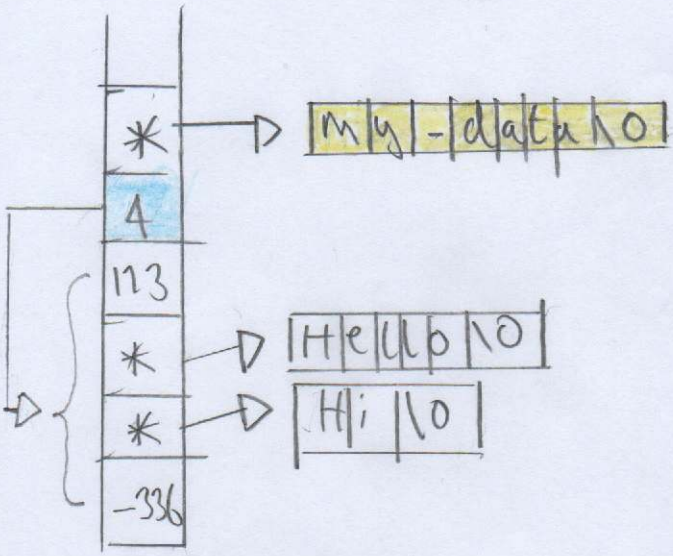
Operand def-i mallocs 17 bytes, copies the top of the run time stack value for the id and pops this off. Then it copies the integer value that's on the top of the run time stack and puts this in the value section (+ 8 bytes).

For strings, it is almost the same:
input file: "Sue jones" user-name def-str



def-str uses the top of run time stack as id, pops it off and uses the top of run time stack (Now Sue jones) as the value. Then pops it off too.

Structures are similar, the run time stack looks like this:
using the input file: -336 int Hi Hello 123 int 4 int my-data def-struct.



My-data1 is at the top of run time stalk, so this is the id. This is popped off. The top of run time stalk is used to malloc this many bytes x8. In this case it is the number 4. This means the 4 nexts below are used to make the structure. This number of values are popped off the stalk and copied into this. Heap allocation.

I wanted to make local variables within functions and their scope (to access) would be only within that function. I needed to do this because I wanted to make this programming language to use recursion. So that means that wherever we call a function it has it's own variables which are local to it and hidden from other functions. I will show you how I managed this later.

I wanted a main function like 'C/C++' so I used '_start' to begin the program. Any variable declared within '_start' would be global scope and available to all of the functions. I didn't want these to be overridden by local variables so I decided to keep ALL global variables unique. That means A variable declared in '_start' can't be declared elsewhere.